Can AI Find What Developers Need? Defining and Measuring Knowledge Availability for AI in Developer Ecosystems

ANONYMOUS AUTHOR(S)

Developers increasingly use AI agents across software development activities, from retrieving technical resources and generating or modifying code to executing tasks through tools. Assisting developers with these tasks requires access to usable technical knowledge. We conceptualize these conditions as knowledge availability for AI and introduce a task-level protocol with eleven indicators covering knowledge sources, acquisition effort, and response properties. We apply the method to 26 categories of software development tasks for AI-accelerated computing, comparing knowledge availability across ecosystems. The analysis distinguishes unclear version information, failures to obtain core instructions, readable but insufficient guidance, and supported main procedures with unavailable references. These findings connect developers' information needs when assessing technical suggestions with knowledge gaps, informing improvements in technical knowledge provision and agents' acquisition and use of knowledge. We contribute a conceptual framework, an inspectable protocol, and a case application for diagnosing the knowledge conditions underlying agents' assistance with development tasks.

CCS Concepts: • **Human-centered computing** → **HCI theory, concepts and models**; • **Software and its engineering** → *Documentation.*

Additional Key Words and Phrases: knowledge availability for AI; developer ecosystems; technical documentation; information foraging; AI agents; measurement methods

1 INTRODUCTION

In traditional software development workflows, developers often need to consult documentation, code examples, search results, and community explanations themselves to decide what to do next. Research on opportunistic programming and developers' information needs shows that finding and interpreting information is part of programming itself [5, 11, 24]. A useful result must fit the current task: a command needs the right options, an API example needs the right version, and an explanation of an error needs enough context to guide investigation. The existence of documentation is therefore only one condition for its usefulness.

Development agents with retrieval and tool-use capabilities introduce another collaborative route through this knowledge environment. A recent study of GitHub projects found adoption of coding agents across projects with different maturity levels and technical characteristics [20]. A developer may ask about an error or a code fragment, or state a desired change or outcome and delegate information seeking, documentation reading, implementation planning, and portions of code modification to an agent. To advance such a task, the agent may search, read, and combine material across official sites, documentation, repositories, and communities before drafting a response, an implementation approach, or a code change. Studies of code-generation tools, conversational programming assistance, and agent support describe opportunities for this support alongside difficulties in understanding and checking generated suggestions [4, 22, 27]. Delegating the search changes how supporting material reaches the developer, while leaving a need to judge whether a suggestion has an identifiable basis, fits the local environment, and requires further information. The agent's access to technical material is one condition for making that basis available for inspection. Measuring this access can therefore identify gaps in the knowledge on which AI-mediated guidance depends.

2026. Manuscript submitted to ACM

Agents can usually produce answers to development questions, but the accuracy and applicability of those answers depend in part on the knowledge they acquire. For example, when implementing a custom operator, an agent may find an official guide without obtaining its procedural content. When investigating an error, it may retrieve a complete page that offers only generic advice to inspect logs. In either case, the agent may continue to answer using prior knowledge or inference while lacking external evidence for specific steps. Even detailed instructions may yield inapplicable suggestions if the toolkit and framework versions they reference are incompatible. Assessment therefore needs to examine not only the answers agents produce but also gaps in their supporting knowledge, identifying concrete opportunities to improve technical resources and the feedback agents provide.

Those building developer ecosystems need to understand not only whether an agent can provide usable technical guidance, but also which knowledge conditions constrain that guidance and where improvements are needed. Similar shortcomings in answers may arise from different conditions: official resources were not discovered, procedural content was not obtained, acquired content lacked task-specific detail, or the basis for selecting a version remained unclear. A usable answer may also rely on alternative sources, masking gaps in official resources. Final answers or task outcomes alone offer limited grounds for distinguishing these conditions. Taking developers' task-specific knowledge needs as a reference, we compare how ecosystems support different development activities and examine sources, acquisition events, and acquired content to identify problems in resource provision, information organization, and complementary sources, informing ecosystem development and agent interaction.

We define **knowledge availability for AI** as the extent to which task-relevant technical knowledge can be discovered, obtained, and assessed for applicability by a specified agent and tool configuration under stated retrieval conditions. The construct is relational: it concerns a task, a knowledge environment, and a means of accessing that environment at a particular time. We operationalize it through an evidence-recording protocol and eleven indicators. The indicators distinguish source conditions, acquisition effort, response checks, and a composite confidence score calculated from M1-M8. The method is intended for professionals such as product managers, designers, technical writers, and software engineers who develop and maintain developer platforms, technical knowledge resources, or AI agents. It aims to help them identify gaps in the discovery, acquisition, and task support provided by technical resources, and locate opportunities to improve knowledge provision and agent feedback. The case application demonstrates diagnostic distinctions for these uses; their value in teams' work requires evaluation.

We ask two research questions. **RQ1:** How can knowledge availability for AI be defined and operationalized so that task-specific access conditions and evidence gaps can be systematically recorded and reviewed? **RQ2:** What patterns of knowledge availability does the method reveal across development tasks, and how can those patterns inform documentation and agent design?

We address these questions through a methodological framework and its application to software development tasks for AI-accelerated computing. The case covers 26 task categories, yielding 52 task-side records. Twenty-five categories compare CANN and CUDA development contexts; one migration category uses ROCm/HIP as the comparison destination and is treated separately. The case application connects task-level measurement to cross-task patterns and design implications. We contribute (1) a construct that distinguishes knowledge conditions from retrieval success and answer correctness; (2) a task-level protocol, indicator definitions, and portable analysis materials; and (3) worked cases and a cross-task synthesis that distinguish different breakdowns and connect them to documentation and agent design.

2 RELATED WORK AND CONSTRUCT BOUNDARIES

2.1 Developer information seeking and documentation

Information foraging theory relates information-seeking behavior to the structure and expected value of available information [19]. Programming studies show how developers interleave searching, learning, and implementation, and how their questions depend on the work they are performing [5, 11, 24]. API-learning difficulties further motivate attention to the explanatory resources surrounding a technical interface [21]. Uddin and Robillard identify ambiguity, incompleteness, and incorrectness as severe API-documentation problems [26]. Aghajani et al. develop a taxonomy of documentation issues from mailing lists, Stack Overflow discussions, issue repositories, and pull requests [1]. Treude and Robillard demonstrate the potential to supplement formal API documentation with useful information extracted from Stack Overflow that the documentation does not contain [25]. Maalej and Robillard further classify knowledge types in API reference documentation [16]. These studies show that developers' information needs are closely tied to their current tasks. We therefore organize the assessment around specific development tasks, examining whether agents can find and obtain the required resources and whether those resources provide the explanations, parameter descriptions, and procedures needed for the task.

Our method does not infer human usability from machine access. Research with developers who are visually impaired likewise finds benefits from AI coding assistance alongside accessibility challenges involving excessive suggestions and context switching, underscoring the need to examine experience in specific usage contexts [7]. A page readable by a browser can be inaccessible to a particular extraction tool; conversely, text readily extracted by an agent can still be difficult for a person to navigate. We examine the conditions of the delegated knowledge route. Claims about developers' time, trust, satisfaction, or task completion require separate evidence.

2.2 AI support for programming

Studies of AI programming assistance distinguish obtaining a suggestion from understanding, adapting, and checking it. Vaithilingam et al. describe difficulties in understanding, editing, and debugging generated code [27]; Barke et al. distinguish acceleration and exploration in programmers' use of code-generating models [4]. Ross et al. examine conversational assistance around code context [22]. In a survey of 410 developers, Liang et al. identify unmet functional or non-functional requirements and difficulty controlling generated output as important barriers to use [13]. Mozannar et al. use a taxonomy of programmer activities to analyze interaction behavior and time costs around code suggestions, motivating assessment of the usage process beyond suggestion generation [17]. In security-related programming tasks, Perry et al. find that participants using an AI assistant produced less secure code while being more likely to believe that their code was secure, indicating a need to distinguish subjective confidence from actual quality [18]. These findings motivate attention to the supporting information available when a suggestion is considered. Our protocol examines whether relevant source passages, version conditions, and procedural details can be obtained and connected to that suggestion. It supplies an account of those knowledge conditions alongside studies of how developers engage with AI assistance.

Agents can interleave reasoning and tool use, and retrieval-augmented generation provides a way to incorporate external information into generation [12, 29]. WebArena, SWE-bench, and SWE-agent evaluate or develop agents in realistic web and software-engineering settings [9, 28, 30]. Their task outcomes are valuable evidence of system performance. Our framework offers a complementary description of the knowledge conditions under which such systems

operate. A failure can involve unavailable evidence, inadequate interpretation of available evidence, or unsuccessful execution; the present method directly addresses the first of these and records information relevant to separating them.

2.3 Retrieval, attribution, and the availability construct

Existing research evaluates AI systems at several levels. ALCE examines the fluency, correctness, and citation quality of generated answers; RAGAs and ARES additionally assess the relevance of retrieved context and the faithfulness of answers to that context [6, 8, 23]. AgentBoard analyzes agents' action processes through measures such as fine-grained task progress, demonstrating the value of process information beyond final success rates [15]. In programming, RepoBench separately evaluates cross-file code retrieval, code completion, and their combined pipeline, distinguishing context acquisition from generation performance [14]. Its focus is completion within code repositories; our method examines whether the official and third-party technical resources needed for development tasks can be discovered, obtained, and provide adequate support. These approaches provide a foundation for understanding system performance. Developing and maintaining a developer ecosystem also requires connecting performance to specific knowledge conditions: whether technical resources are readily discoverable, whether the required content is actually obtained, whether it supplies procedural detail, and whether versions have clear selection and compatibility guidance. We organize task-level measurement around these relationships to distinguish conditions calling for better acquisition paths, fuller content, or clearer version relations.

The distinctions in Table 1 position the construct. Availability can be necessary for an evidence-grounded answer without being sufficient for correctness. An agent can misinterpret an available source. It can also produce a correct answer from prior knowledge without obtaining any external evidence. These outcomes should remain distinguishable.

Table 1. Construct boundaries and neighboring objects of assessment.

Object of assessment	Central question	Relationship to this method
Retrieval success	Was a relevant result returned?	One acquisition condition; not proof that required content was obtained.
Documentation usability	Can intended readers navigate and understand the material?	A related human-facing property requiring its own evaluation.
Knowledge availability for AI	What task-relevant knowledge was obtainable under the recorded conditions?	The construct operationalized here.
Answer correctness and attribution	Is the response correct, and do its sources support its claims?	A downstream evaluation that may use the acquired evidence.
Execution success	Do the proposed actions work in the target environment?	Requires execution or other independent outcome evidence.

3 METHODOLOGICAL FRAMEWORK

3.1 Unit of analysis and task requirements

The unit is a **task-side acquisition episode**: a development objective pursued in a particular technical ecosystem using a stated agent and retrieval configuration. A task specification records the desired outcome, starting artifacts,

constraints, version dependencies, and evidence required to support a next action. For example, model conversion may require a conversion command, an input-shape specification, a target-device setting, and a way to check the generated artifact. These requirements guide inspection; a long page is not automatically adequate.

Task selection and sampling should be explicit and follow the purpose of the evaluation [10]. An ecosystem assessment may seek coverage across workflows; a documentation-team audit may focus on recurring support questions. Neither establishes the population frequency of those tasks without additional sampling evidence. For cross-ecosystem use, analysts record differences in starting conditions and task scope rather than assuming that similar names make tasks equivalent. The framework can also be applied within one ecosystem, across versions, or across retrieval configurations, provided the comparison conditions are made explicit.

Figure 1 locates the measurements in an observable tool-use loop. It is an analytic model, not a reconstruction of hidden model reasoning: the agent searches for candidate sources, selects URLs, fetches content or records a failure, then integrates evidence and assesses task adequacy and version applicability. When evidence is insufficient but remains searchable, the loop returns to query refinement. Model prior is registered as a separate branch that need not produce an observable source; it is therefore not automatically treated as traceable evidence.

3.2 Knowledge sources and task support

The framework examines three knowledge sources: official material, third-party material, and model prior knowledge. Official and third-party material are distinguished by publisher identity. For example, documentation, repositories, and blogs published by a vendor belong to the official channel, while tutorials and practical accounts from independent authors belong to the third-party channel. The hosting platform alone does not determine source ownership.

For official and third-party material, the framework examines whether the agent can find relevant sources, obtain their content, and use that content to support the current task. Specific checks concern whether the material includes the required procedures, parameter explanations, or diagnostic information, and whether applicable versions and component compatibility are clear. For model prior knowledge, support for the current task is estimated and presented separately from the external material actually acquired.

Considering the three sources together makes it possible to examine which sources supply the knowledge required for a task, which requirements remain unsupported, and whether sources complement one another. For example, when official content is not obtained, does third-party material provide usable instructions? When several sources contain relevant information, are their versions and compatibility statements consistent? Acquisition counts and failed attempts describe the effort required to obtain this knowledge.

3.3 An evidence record

The evidence record connects each judgment to its basis. Its minimum fields are the task question and starting conditions; agent and tool configuration when known; collection time; queries and selected URLs; publisher and source role; returned content or a relevant excerpt; access outcome; applicable versions; task requirements supported by that content; acquisition counts; and the coding decision with a rationale. A field also identifies whether an entry is directly recorded, subsequently coded, estimated, or unavailable.

This distinction prevents a common reporting error: transforming an interpretation into an observation merely because a script assigns it a number. A logged extraction failure is an observation of the tool-source interaction. Labeling an explanation as sufficient is a judgment against task requirements. Estimating a model's familiarity with a toolkit is an inference unless a separate test supports it. Numerical coding does not erase these differences.

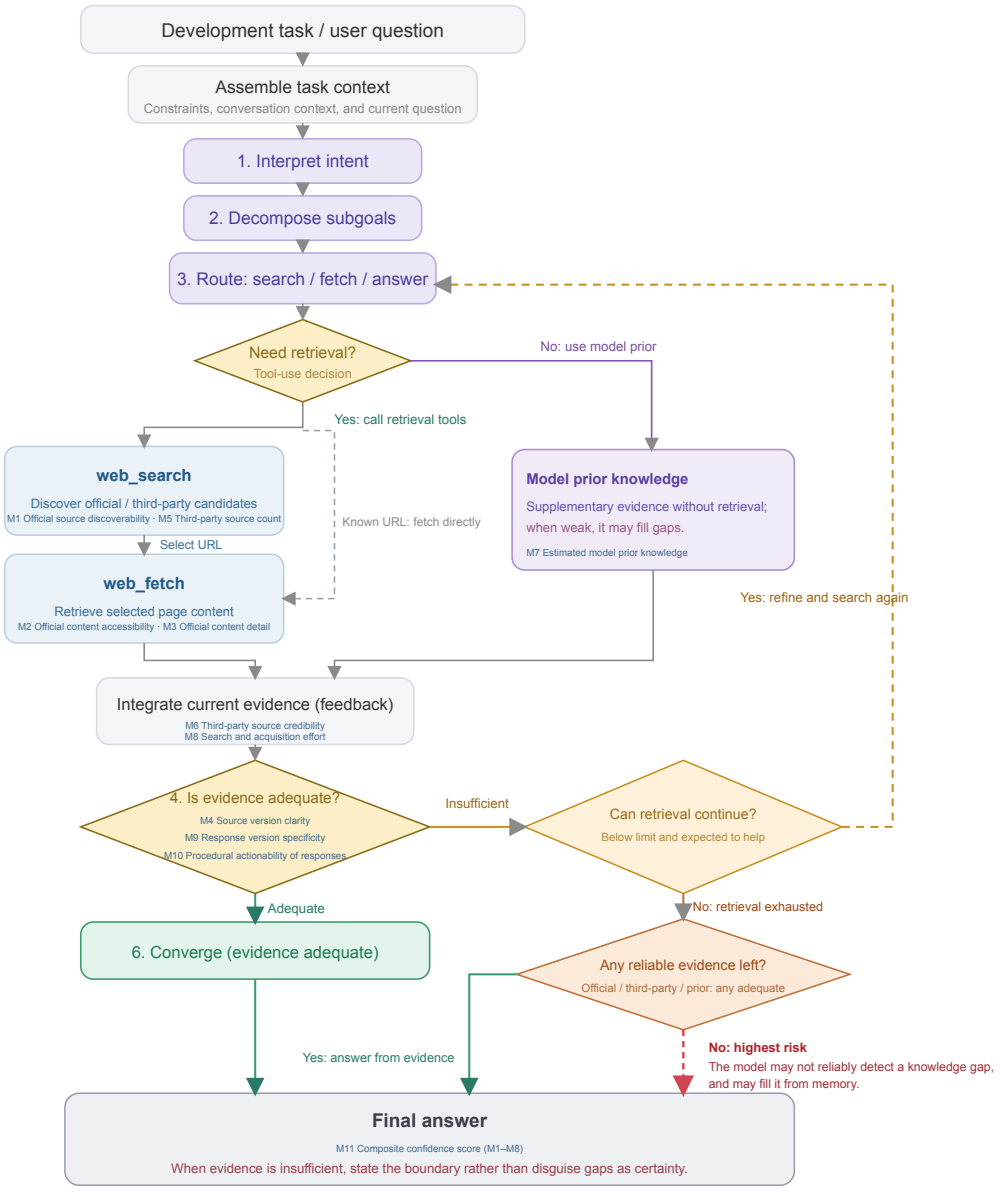


Fig. 1. An agent starts from a task and context, interprets intent, decomposes subgoals, and routes to retrieval or model prior; a retrieval branch searches, selects URLs, fetches content or fetches a known URL directly, then integrates evidence and assesses adequacy. Insufficient evidence can trigger retry; exhausted retrieval distinguishes whether reliable evidence remains.

4 OPERATIONALIZATION AND MEASUREMENT PROTOCOL

4.1 Eleven indicators and their evidential roles

The method examines three knowledge sources: official material, third-party material, and model prior knowledge. It assesses discovery, acquisition, and source quality, and connects knowledge acquisition to response generation through measures of retrieval effort and response properties. Table 2 presents the eleven indicators and their scoring criteria.

Table 2. The eleven indicators and the evidence each can support.

ID	Indicator	Evidence and interpretation
M1	Official source discoverability	Measures the ease of finding task-relevant official material using result position, search rounds, and the need for query refinement.
M2	Official content accessibility	Assesses the retrieval of task-relevant official content using the selected page's fetch category, returned body content, and access barriers.
M3	Official content detail	Measures the detail of task-relevant official material using its coverage level and the actionability of commands and code. Content that cannot be retrieved is marked as blocked.
M4	Source version clarity	Measures how clearly official material enables selection of an applicable version, considering coexisting versions, compatibility matrices, and device–framework compatibility conditions.
M5	Third-party source count	Measures the number of third-party sources found during retrieval, with scores assigned according to the recorded source count.
M6	Third-party source credibility	Assesses third-party credibility from source type, with adjustments for publication age, content consistency, and distribution across source platforms.
M7	Estimated model prior knowledge	Estimates the support that existing model knowledge provides for a task, using model self-assessment adjusted for the pace of relevant technical change.
M8	Search and acquisition effort	Measures retrieval effort from search rounds, fetch attempts, and failed fetches. Lower acquisition cost receives a higher score.
M9	Response version specificity	Measures how precisely a response identifies applicable versions and component combinations, ranging from exact specifications to unresolved version choices.
M10	Procedural actionability of responses	Assesses the operational completeness of response commands and steps, distinguishing direct use, parameter substitution, minor adaptation, and skeletal guidance.
M11	Composite confidence score	Combines support from official material, third-party material, and model prior knowledge, with adjustments for version clarity and retrieval effort, using the formula below.

M1–M7 describe support from the three knowledge sources, while M8 captures the effort required to acquire that knowledge. M9 and M10 assess response version specificity and procedural actionability. M11 is calculated from M1–M8, with response properties reported separately. Together, the individual indicators and composite confidence characterize each task's knowledge support and help identify areas for improvement.

4.2 Applying the protocol

First, define the task and the recording boundary. State the development outcome, starting artifacts, and requirements against which adequacy will be assessed. Record the agent, tools, language, date, and context policy. Specify whether the unit includes a fresh acquisition, reused observations, or both. Set a retrieval budget or another observable stopping criterion appropriate to the application; an analyst must not infer an agent's internal reasons for stopping from the number of queries alone.

For cross-ecosystem comparisons, each task comprises a pair of questions organized around a shared development intent, with wording adapted to the tools, APIs, and development workflows of the two ecosystems. Pairing aims for comparable development goals, task scope, and levels of requested guidance, so that both questions address the same type of development need. For example, both may ask how to install a toolchain, establish version compatibility, or locate a memory error, while using the terminology and implementation paths of their respective ecosystems. Differences in starting conditions or task scope should be stated explicitly and considered when interpreting the results.

Second, record discovery and acquisition. Preserve queries, candidate sources, selected URLs, and read outcomes. Inspect official materials and record the community alternatives actually encountered, whether from mixed search results or subsequent searches. Keep a failed read separate from a failed search. When a mirror or another version supplies the required content, link it to the original attempt so that the workaround remains visible.

Third, assess task support and applicability. Map acquired material to the task requirements. Distinguish a returned document with missing essential content from a document that was never returned. Record stated versions and compatibility dependencies, including unresolved local facts. A version number in a URL is a clue, not by itself a verified compatibility relation.

Fourth, score from observations. Apply published anchors and explain the score assigned to each observation, and mark estimates and missing fields. Content acquisition status has four states: core content obtained; partial content obtained; required content not obtained; and unknown. Record access route separately as original entry, alternative entry, or unknown. Thus a complete mirrored body remains core content obtained even when it arrived through an alternative entry. Adequacy can use ordered anchors ranging from a generic fragment, through an overview and a usable main path, to detailed task-relevant reference material. The coding rationale identifies the specific requirements covered; it should not rely only on page length.

Fifth, inspect the instructions and report the profile. Identify the response or guidance used to assess M9 and M10. Map its key steps and version claims to acquired evidence, and record contradictions, source dependencies, and unsupported steps. Report the composite-confidence formula, missing-data treatment, and calculation assumptions. The accompanying worked example connects requirements to events, sources, acquired evidence, and scores, with raw call counts shown alongside the M8 effort score.

The method's novelty lies in this connected specification of the unit, evidence record, indicator roles, and diagnostic interpretation. Search rank, reading success, and version labels are familiar observations. Their value here comes from relating them to the requirements of one development task and explaining how an observation becomes a diagnosis.

4.3 Scoring rules and composite confidence

M1–M10 are mapped to ordinal scores from 1 to 5 using their scoring rules, with M3 marked as blocked when its body content is not obtained. M11 combines M1–M8 into a composite score on a 0–1 scale. We describe the anchors by knowledge source, acquisition effort, response properties, and composite confidence. Appendix A specifies decision priorities and adjustments; task records provide the corresponding observations.

Official material (M1–M4). Discoverability combines result position, search rounds, and query refinement; body accessibility considers robots restrictions followed by page delivery and observed body returns. Grade 3 denotes partial retrieval of task-relevant body content, with detailed conditions in Appendix A. Body detail combines task coverage with the actionability of commands and code, while source version clarity considers version identifiers and compatibility conditions. Figure 2 shows the four sets of anchors: the lowest M3 grades distinguish whether task-specific details are present, while M4 distinguishes missing identifiers from listed versions with unclear compatibility. M3 is marked blocked when its body cannot be acquired.

Score		1	2	3	4	5
M1	Official source discoverability	Still not found	Refine query / rank >10	Multiple rounds / rank 7–10	First round: 2–6	First round: 1
M2	Official content accessibility	robots restriction	SPA body blocked	Partial body retrieval	SSR body retrievable	Static body retrievable
M3	Official content detail	No task details	A few task fragments	Overview / main path, no code	Main path + code / full ref., no code	Full reference + commands/code
M4	Source version clarity	Version IDs missing	≥3 versions listed pairing unclear	Chip–framework pairing / other	Support matrix	Version irrelevant / at most one

Fig. 2. Scoring anchors for official material (M1–M4); M2 grade 3 denotes partial retrieval of task-relevant body content.

Third-party material (M5–M6). M5 scores the number of distinct third-party sources. M6 starts from the mean source-type baseline, adjusted for consistency, publication age, and platform distribution.

Model prior knowledge (M7). M7 starts with a model self-rating from 1 to 5 of how well its existing knowledge supports the current task, then accounts for the risk of outdated knowledge as the relevant technology changes. Stable, moderate, and rapid change incur deductions of 0, 0.25, and 0.5, respectively. The adjusted value is rounded using the rule in Appendix A and bounded between 1 and 5. Figure 3 shows the anchors before adjustment.

Retrieval and acquisition effort (M8). M8 calculates weighted effort from search rounds, fetch calls, and failed fetches:

$$c = r + 0.5f + 4e$$

Here, r denotes search rounds, f denotes all fetch calls, including failed attempts, and e denotes failed fetches. One search round is the basic effort unit. Fetching an already located page primarily acquires and checks evidence and has a more defined scope than searching again; each fetch is therefore assigned half a search-round unit, with a coefficient of 0.5. A failed fetch may entail identifying an empty body, revising a query or seeking a mirror, fetching again, and assessing the returned content. Each failure therefore incurs an additional penalty of four effort units to emphasize

the burden of acquisition barriers. The coefficients 0.5 and 4 are relative effort weights specified by the protocol, rather than measured time ratios; subsequent searches and fetches are still counted in r and f, respectively. Weighted effort is mapped to a score from 1 to 5 using the thresholds in Figure 3, with lower effort receiving a higher score. Appendix A gives the complete mapping.

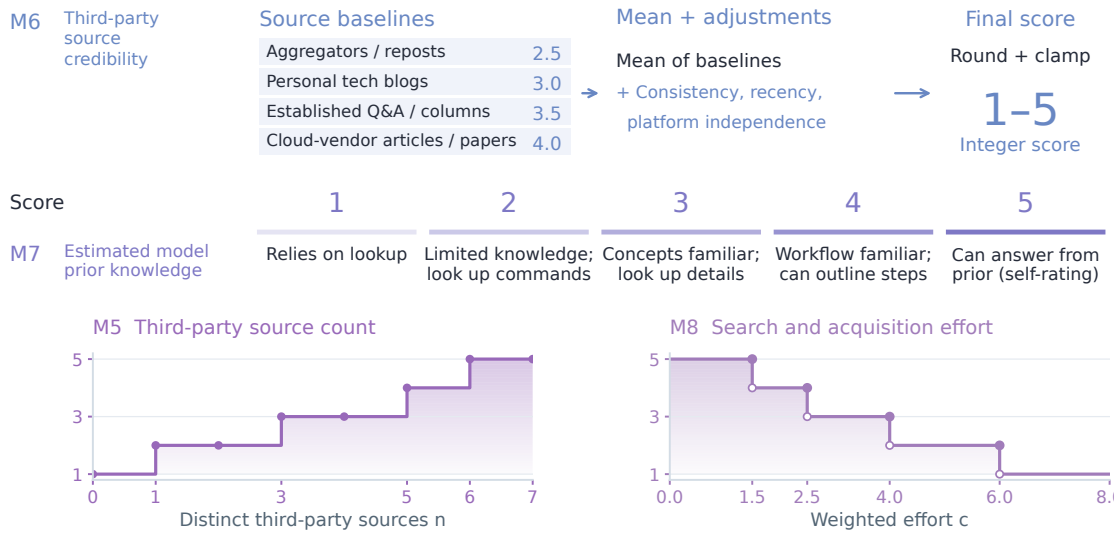


Fig. 3. Scoring components for M5–M8: M7 self-ratings, M5 and M8 mappings, and M6 source-type baselines with adjustments. Appendix A specifies adjustment values.

Response properties (M9–M10). M9 examines whether a response specifies versions and component combinations applicable to the task; M10 examines the operational conditions of its commands and steps. Figure 4 distinguishes the lower grades: version clues with unresolved applicability score above no specific version information, and a main workflow outline lacking key details scores above isolated fragments. These separately reported properties connect source conditions to the guidance produced.

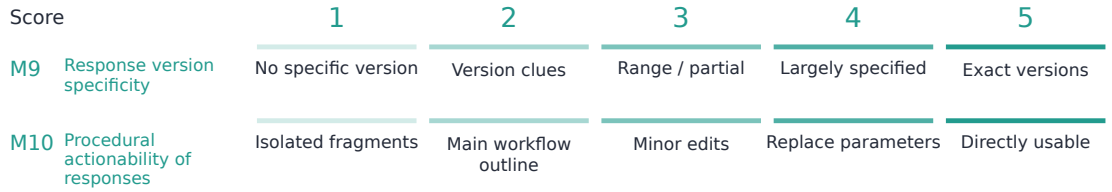


Fig. 4. Scoring anchors for answer version specificity (M9) and step actionability (M10).

Composite confidence (M11). M11 is calculated from M1–M8 in three steps. Each indicator score s_i is first normalized as $n(s_i) = s_i/5$.

Step 1: Calculate support from each knowledge channel. Official material must be discoverable, retrievable, and sufficiently detailed for the task. The normalized scores for M1, M2, and M3 are therefore multiplied to obtain official-channel support O. Third-party support C combines source quantity M5 and credibility M6; model prior-knowledge support P uses M7:

$$O = n(s_1)n(s_2)n(s_3), \quad C = n(s_5)n(s_6), \quad P = n(s_7).$$

Multiplication within a channel makes shortcomings in any component reduce its support score. When M3 is marked as blocked, official-channel support is set to $O = 0$.

Step 2: Combine support across channels.

$$K = 1 - (1 - O)(1 - C)(1 - P).$$

This combination represents the potential for knowledge channels to complement one another: third-party material or model prior knowledge may still provide support when official material is insufficient. K is a knowledge-support score constructed from ordinal ratings, rather than a calibrated probability of answer correctness.

Step 3: Adjust for version clarity and acquisition effort.

$$I = K\,[0.7 + 0.3n(s_4)]\,[0.9 + 0.1n(s_8)]\,.$$

As source version clarity M4 and retrieval-and-acquisition-effort score M8 increase, the two adjustment factors approach 1, reducing the discount applied to the knowledge-support score. The coefficients 0.3 and 0.1 control the adjustment magnitudes for version clarity and effort, respectively, assigning a larger discount to unclear versions than to high acquisition effort at equivalent normalized ratings. The final score is displayed using the bands in Figure 5. M9 and M10 are reported separately as response properties and do not enter M11.



Fig. 5. Composite-confidence bands. Lower bounds are inclusive; the highest band includes 1.

The task matrix presents the indicator scores, while the access-profile figure shows the corresponding content-acquisition outcomes to support interpretation. Appendix A specifies the calculations; the supplement provides task-level worked examples.

5 CASE STUDY OF THE CANN AND CUDA DEVELOPER ECOSYSTEMS

5.1 Task coverage and comparison scope

We conducted the development-task audit on June 10-11, 2026, covering environment setup, operator development, training, inference and deployment, performance analysis, debugging, and migration. Examples include model conversion, custom-operator integration, distributed initialization, version compatibility, and memory-error investigation. The task set sought workflow coverage; it was not sampled to estimate how often developers encounter these needs. Developer-role groupings organize task scenarios around the typical knowledge needs of different roles.

The CANN/CUDA comparison organized task-question pairs according to these principles. Table 3 presents three representative pairs and their shared development intents; the question-pair table in the supplement provides the complete questions.

Table 3. Examples of question pairs organized around shared development intents. Questions are translated and condensed for presentation; the complete Chinese questions are provided in the supplement.

Task and shared development intent	CUDA question summary	CANN question summary
J: Install a toolchain and configure the environment	After installing the CUDA toolkit on Ubuntu, which environment variables need to be configured?	After installing the CANN toolkit on Ubuntu, which environment script and variables are required?
I: Establish component version compatibility	How can compatibility among PyTorch, the CUDA toolkit, cuDNN, and the driver be checked?	How can compatibility among CANN, torch_npu, firmware and drivers, and PyTorch be checked?
X: Locate an operator memory-access violation	How can compute-sanitizer locate an out-of-bounds access in a CUDA kernel?	How can tools such as msSanitizer locate an out-of-bounds access in an Ascend operator?

Implementation steps and some starting conditions may differ between the two sides. For example, task A's CUDA question begins with a PyTorch model and asks for a TensorRT deployment path, whereas its CANN question starts with an ONNX model and asks for an ATC conversion. These scope differences are considered when interpreting acquisition effort and content completeness.

Task G is a distinct migration analogy. Its comparison question concerns moving CUDA code to AMD ROCm/HIP and its official material comes from AMD. It is included in the 26-task set as a worked migration case, but is excluded from CANN/CUDA aggregate comparisons, leaving 25 pairs and 50 task-side records for those summaries. Its separate treatment demonstrates why source and comparison identity belong in the measurement protocol.

5.2 Task execution and data analysis

We used a process log to record task questions, query strings, source descriptions, read outcomes, and scoring rationales. Scoring functions converted the observation fields into an interactive task matrix and analytic visualizations. Some early read assessments reused observations already obtained in the same session.

Tasks were conducted in four batches. The first two covered A–D and E–H; the later two covered I–S and T–Z through parallel subagents. Structured observations were consolidated in the main session, where official and third-party source ownership was checked. Task questions were linked to their original assignments, and wording for the later batches was added to the process log from those assignments.

Tasks used Claude (model identifier `claude-opus-4-8`) with web-search and web-reading tools. Search returned candidate sources, and the reading tool summarized HTML from selected URLs. The analysis combined recorded queries, tool-returned content and summaries, structured observations, and scoring rationales. M9 and M10 assessed

version specificity and procedural actionability using the guidance produced during the tasks and its corresponding records. The supplement links tasks, sources, and scoring rationales.

We used a consistent evaluation framework, eleven indicators, and scoring rules to acquire and assess knowledge for each development task.

The analysis proceeds at three levels. First, we compare indicator scores within corresponding task categories to locate differences in resource discovery, content acquisition, content detail, version clarity, and guidance properties. Second, we describe indicator distributions using means ($\bar{x}$), standard deviations (SD), and counts at each score level, and summarize composite confidence by workflow. Third, we interpret score differences using the corresponding acquisition records and returned content, comparing content support under the same acquisition state and knowledge conditions across tasks within a workflow. M3 means and standard deviations include only records for which content detail can be assessed; blocked-content records are reported separately. These statistics describe the selected task set and do not support population-level inference about the ecosystems.

Beyond workflow comparisons, we group the 25 task pairs by development activity into getting started and environment setup, training and routine use, and operators and in-depth development. Both ecosystems use the same task assignments to compare knowledge-support distributions across these scenarios; Table 6 reports the assignments and descriptive statistics. G, whose comparison side is ROCm/HIP, remains a separate case and is excluded from CANN/CUDA aggregates. Scenario grouping organizes the task comparison, while individual indicator scores and corresponding materials explain the specific knowledge conditions.

5.3 Task-level assessment results

The main comparison includes 25 CANN tasks: core content was obtained in 22 tasks, partial content in two, and no core content in one. Core content was obtained in all 25 corresponding CUDA tasks. Figures 6–8 present all eleven indicator scores and group tasks by environment and installation, operator development, training, inference and deployment, performance and optimization, debugging, and migration. G is shown as a separately marked CANN/ROCm-HIP migration analogy and is excluded from CANN/CUDA summaries.

Figures 6–8 present the ordinal scores for M1–M10 and composite confidence calculated from M1–M8. Figure 9 adds content-acquisition states to support interpretation of the corresponding scores.

Across the 25 task pairs, mean composite confidence is $\bar{x} = 0.731$ for CANN and $\bar{x} = 0.922$ for CUDA. Indicator-level summaries show differences of varying sizes in content detail, source version clarity, search and acquisition effort, and procedural actionability of responses. Table 4 reports the means and standard deviations of these four indicators. The findings below interpret the corresponding knowledge conditions using score distributions and specific resources.

A. Official-source conditions

Indicator scores by workflow. Each task row has CANN and CUDA columns; G uses ROCm/HIP in the second column.

Task		M1 Official find		M2 Official access		M3 Official detail		M4 Source version	
		CANN	CUDA	CANN	CUDA	CANN	CUDA	CANN	CUDA
Environment and installation									
I	Version compatibility	3	3	4	5	4	5	4	4
J	Installation	5	4	4	5	5	5	2	3
K	Container setup	5	5	5	5	5	4	4	5
Operator development									
C	Library selection	4	5	3	5	3	5	3	5
D	Custom operator	2	5	2	5	—	5	2	3
L	Operator accuracy	2	2	4	5	5	5	2	5
M	Dynamic shapes / tiling	2	5	4	5	4	5	4	4
N	Operator fusion	5	2	4	5	5	4	2	5
O	Framework integration	2	5	4	5	5	5	3	3
Training									
F	Distributed training	4	5	4	5	4	5	2	3
P	Mixed precision	3	5	4	5	4	5	4	5
Q	Memory / OOM	5	5	4	5	5	5	4	5
R	Training convergence	3	3	4	5	5	4	3	5
Inference and deployment									
A	Model conversion	4	5	4	5	4	5	2	4
H	Quantization	3	5	4	5	3	5	2	3
S	Inference serving	5	5	4	5	4	5	3	4
T	Dynamic inference	3	3	4	5	5	5	2	5
Performance and optimization									
B	Profiling	4	4	4	5	4	5	2	4
U	Memory / occupancy	4	5	4	5	4	5	5	5
V	Compute-transfer overlap	4	5	4	5	4	5	2	5
W	Multi-device communication	2	5	4	5	3	4	3	4
Debugging									
E	Error-code diagnosis	3	4	4	5	2	5	2	5
X	Memory-error diagnosis	5	5	4	5	5	5	2	3
Migration									
Y	Chip migration	2	5	3	5	4	5	3	4
Z	Programming concepts	5	5	4	5	5	4	5	5
Migration analogy (not in CANN/CUDA summaries)									
G	Migration analogy	4	5	4	5	4	5	4	3

M1–M10 are ordinal scores (1–5; higher is more favorable; M8 is an inverse effort score). "—" = unobserved official content detail.

* M7 estimates model prior knowledge. † M11 is composite confidence from M1–M8. For G, CUDA = ROCm/HIP; it is excluded from CANN/CUDA summaries.

Fig. 6. Full indicator matrix (1/3): M1–M4 scores for official discoverability, official content accessibility, official content detail, and source version clarity, organized by workflow with CANN and CUDA columns for each task. G uses ROCm/HIP in the second column.

Table 4. Descriptive statistics for four indicators, scored from 1 to 5. *SD* denotes the sample standard deviation. For M3, CANN has $n = 24$, excluding the blocked-content record D, and CUDA has $n = 25$. Both sides have $n = 25$ for the other indicators. Higher M8 scores indicate lower search and acquisition effort. Means and standard deviations describe the ordinal score distributions.

Indicator	CANN $\bar{x}$ (*SD*)	CUDA $\bar{x}$ (*SD*)
M3 Official content detail	4.21 (0.83)	4.80 (0.41)
M4 Source version clarity	2.88 (1.01)	4.24 (0.83)
M8 Search and acquisition effort	3.24 (1.36)	3.88 (1.51)
M10 Procedural actionability of responses	4.00 (0.76)	4.72 (0.68)

B. Third-party, prior, and acquisition conditions

Indicator scores by workflow. Each task row has CANN and CUDA columns; G uses ROCm/HIP in the second column.

Task		M5 3P count		M6 3P credibility		M7 Prior estimate*		M8 Search / access effort	
		CANN	CUDA	CANN	CUDA	CANN	CUDA	CANN	CUDA
Environment and installation									
I	Version compatibility	3	2	4	4	2	4	3	3
J	Installation	3	3	4	4	4	5	5	5
K	Container setup	2	3	2	4	3	5	1	4
Operator development									
C	Library selection	3	5	4	4	3	5	5	5
D	Custom operator	3	4	2	4	2	5	1	5
L	Operator accuracy	3	2	4	4	3	4	4	1
M	Dynamic shapes / tiling	2	2	4	4	2	5	3	5
N	Operator fusion	2	2	3	4	2	4	5	1
O	Framework integration	3	2	5	4	3	5	1	3
Training									
F	Distributed training	3	3	3	4	3	4	5	5
P	Mixed precision	3	3	4	4	4	5	2	4
Q	Memory / OOM	3	3	4	4	3	5	3	2
R	Training convergence	3	3	4	3	3	5	3	4
Inference and deployment									
A	Model conversion	4	4	4	4	3	4	4	5
H	Quantization	4	3	3	4	3	4	4	5
S	Inference serving	2	3	4	4	2	4	4	5
T	Dynamic inference	3	2	5	4	3	5	3	1
Performance and optimization									
B	Profiling	3	4	3	4	3	4	4	5
U	Memory / occupancy	3	3	4	4	3	5	4	5
V	Compute-transfer overlap	3	3	4	4	3	5	4	4
W	Multi-device communication	3	3	4	4	3	4	3	5
Debugging									
E	Error-code diagnosis	3	4	3	4	2	4	4	5
X	Memory-error diagnosis	2	3	4	4	2	4	4	5
Migration									
Y	Chip migration	3	3	4	4	3	5	1	4
Z	Programming concepts	3	3	4	4	4	5	1	1
Migration analogy (not in CANN/CUDA summaries)									
G	Migration analogy	3	3	4	4	3	4	5	5

M1–M10 are ordinal scores (1–5; higher is more favorable; M8 is an inverse effort score). "—" = unobserved official content detail.

* M7 estimates model prior knowledge. † M11 is composite confidence from M1–M8. For G, CUDA = ROCm/HIP; it is excluded from CANN/CUDA summaries.

Fig. 7. Full indicator matrix (2/3): M5–M8 scores for third-party source count, third-party source credibility, estimated model prior knowledge, and search and acquisition effort, in the same workflow and task order.

6 FINDINGS: KNOWLEDGE CONDITIONS ACROSS DEVELOPMENT TASKS

Task-level comparisons reveal three connected findings. First, the ecosystems show different distributions of knowledge support across development scenarios. Second, differences in specific tasks involve official-source discovery, content delivery, detail, and version organization. Finally, third-party material and estimates of model prior knowledge also vary by task and require examination alongside official resources. We present scenario comparisons, specific knowledge gaps, and complementarity among sources in that order.

6.1 Knowledge-support distributions differ between ecosystems across development scenarios

Across workflows, CANN's mean composite confidence is $\bar{x} = 0.799$ for environment and installation, $\bar{x} = 0.658$ for operator development, and $\bar{x} = 0.646$ for debugging; the corresponding CUDA means are $\bar{x} = 0.904$, $\bar{x} = 0.891$, and

C. Response checks and composite confidence

Indicator scores by workflow. Each task row has CANN and CUDA columns; G uses ROCm/HIP in the second column.

Task		M9 Response version		M10 Actionability		M11 Confidence†	
		CANN	CUDA	CANN	CUDA	CANN	CUDA
Environment and installation							
I	Version compatibility	5	5	5	5	0.73	0.85
J	Installation	4	4	5	5	0.80	0.88
K	Container setup	4	4	5	5	0.86	0.98
Operator development							
C	Library selection	4	5	4	5	0.75	1.00
D	Custom operator	2	5	3	5	0.41	0.88
L	Operator accuracy	3	4	4	4	0.69	0.84
M	Dynamic shapes / tiling	4	4	4	5	0.63	0.94
N	Operator fusion	4	4	4	4	0.74	0.83
O	Framework integration	4	4	5	5	0.72	0.84
Training							
F	Distributed training	3	4	3	5	0.72	0.88
P	Mixed precision	4	5	4	5	0.83	0.98
Q	Memory / OOM	3	4	4	5	0.86	0.94
R	Training convergence	3	4	4	4	0.75	0.98
Inference and deployment							
A	Model conversion	3	5	4	5	0.75	0.94
H	Quantization	4	4	5	5	0.69	0.88
S	Inference serving	4	4	4	5	0.74	0.94
T	Dynamic inference	4	5	4	5	0.72	0.92
Performance and optimization							
B	Profiling	3	4	3	5	0.70	0.93
U	Memory / occupancy	4	4	4	4	0.88	1.00
V	Compute-transfer overlap	4	5	4	5	0.72	0.98
W	Multi-device communication	4	5	4	5	0.70	0.92
Debugging							
E	Error-code diagnosis	3	4	3	5	0.55	0.99
X	Memory-error diagnosis	4	4	5	5	0.74	0.88
Migration							
Y	Chip migration	4	5	4	5	0.67	0.92
Z	Programming concepts	2	2	2	2	0.90	0.92
Migration analogy (not in CANN/CUDA summaries)							
G	Migration analogy	4	4	4	5	0.84	0.88

M1–M10 are ordinal scores (1–5; higher is more favorable; M8 is an inverse effort score). "—" = unobserved official content detail.

* M7 estimates model prior knowledge. † M11 is composite confidence from M1–M8. For G, CUDA = ROCm/HIP; it is excluded from CANN/CUDA summaries.

Fig. 8. Full indicator matrix (3/3): M9–M11 values for response version specificity, procedural actionability of responses, and the composite confidence score from M1–M8, in the same workflow and task order.

$\bar{x} = 0.933$ (Table 5). Debugging has the largest between-ecosystem difference, at 0.287. The comparison directs attention to specific development activities: in this task set, CANN provides stronger knowledge support for environment configuration than for operator development and debugging.

Table 5. Workflow means of the composite confidence score for 25 task pairs. Differences are calculated as CUDA minus CANN using unrounded means, then rounded to three decimal places; G is excluded.

Workflow	Tasks	CANN $\bar{x}$	CUDA $\bar{x}$	Difference
Environment and installation	3	0.799	0.904	0.106
Operator development	6	0.658	0.891	0.233
Training	4	0.791	0.945	0.154

Workflow	Tasks	CANN $\bar{x}$	CUDA $\bar{x}$	Difference
Inference and deployment	4	0.722	0.920	0.198
Performance and optimization	4	0.752	0.957	0.205
Debugging	2	0.646	0.933	0.287
Migration	2	0.787	0.921	0.133

Table 6. Mean composite confidence and sample standard deviation for 25 task pairs grouped by development scenario. Getting started and environment setup comprises A, H, I, J, K, S, T, Y, Z; training and routine use comprises E, F, P, Q, R; operators and in-depth development comprises B, C, D, L, M, N, O, U, V, W, X. Both ecosystems use identical assignments, excluding G.

Development scenario	Task pairs	CANN $\bar{x}$ (SD)	CUDA $\bar{x}$ (SD)
Getting started and environment setup	9	0.762 (0.078)	0.915 (0.038)
Training and routine use	5	0.743 (0.121)	0.953 (0.045)
Operators and in-depth development	11	0.699 (0.113)	0.914 (0.062)

Scenario grouping further shows CANN's mean composite confidence decreasing from $\bar{x} = 0.762$ for getting started and environment setup to $\bar{x} = 0.743$ for training and routine use and $\bar{x} = 0.699$ for operators and in-depth development. CUDA's corresponding means are $\bar{x} = 0.915$, $\bar{x} = 0.953$, and $\bar{x} = 0.914$, without the same descending pattern. This comparison identifies weaker knowledge support for in-depth development in CANN, allowing the assessment to produce specific insights into how an ecosystem supports different development activities.

Cases within the groups clarify the knowledge these scenarios require. CANN's tiling task M and custom-operator task D score 0.628 and 0.410, respectively. Memory-access and utilization task U, also in the in-depth development group, scores 0.881 and obtained official best-practice material. Error-code task E in the training and routine-use group scores 0.554; its acquired official explanation lacked diagnostic detail. We next examine acquisition and content records to identify the knowledge-provision issues involved in these scenario differences.

6.2 Official-source discovery, acquisition, content, and version organization reveal distinct gaps

Official-source discoverability also differs. Across the 25 task pairs, mean M1 is $\bar{x} = 3.56$ for CANN and $\bar{x} = 4.40$ for CUDA, with 12 and five tasks, respectively, using two search rounds. In task A, the first official CANN result appeared at rank 4, compared with rank 1 for the corresponding CUDA task. These differences concern the position of official material in task-related search results and the number of rounds used in the retrieval process.

After official material is found, the next question is whether the required body content is actually obtained. In the main comparison, 22 of the 25 CANN tasks obtained core content, two obtained partial content, and one did not obtain core content; all 25 CUDA tasks obtained core content. Most tasks acquired official material, while acquisition obstacles were concentrated in a few paths. Those paths require examination against task requirements to identify the missing procedural material.

Task D requires creating an Ascend C operator and integrating it with a framework, with implementation code, configuration-field explanations, and compilation and deployment steps. In task D, when the agent read the official

Access-profile detail

Content-acquisition states complement the M2 accessibility scores in Figures 2–4.

Task	Content access		Official content detail		Source version clarity	
	CANN	CUDA	CANN	CUDA	CANN	CUDA
Environment and installation						
I Version compatibility	C	C	4	5	4	4
J Installation	C	C	5	5	2	3
K Container setup	C	C	5	4	4	5
Operator development						
C Library selection	P	C	3	5	3	5
D Custom operator	N	C	—	5	2	3
L Operator accuracy	C	C	5	5	2	5
M Dynamic shapes / tiling	C	C	4	5	4	4
N Operator fusion	C	C	5	4	2	5
O Framework integration	C	C	5	5	3	3
Training						
F Distributed training	C	C	4	5	2	3
P Mixed precision	C	C	4	5	4	5
Q Memory / OOM	C	C	5	5	4	5
R Training convergence	C	C	5	4	3	5
Inference and deployment						
A Model conversion	C	C	4	5	2	4
H Quantization	C	C	3	5	2	3
S Inference serving	C	C	4	5	3	4
T Dynamic inference	C	C	5	5	2	5
Performance and optimization						
B Profiling	C	C	4	5	2	4
U Memory / occupancy	C	C	4	5	5	5
V Compute-transfer overlap	C	C	4	5	2	5
W Multi-device communication	C	C	3	4	3	4
Debugging						
E Error-code diagnosis	C	C	2	5	2	5
X Memory-error diagnosis	C	C	5	5	2	3
Migration						
Y Chip migration	P	C	4	5	3	4
Z Programming concepts	C	C	5	4	5	5
Migration analogy (not in CANN/CUDA summaries)						
G Migration analogy [CANN / ROCm-HIP]	C	C	4	5	4	3

Content status: C = core obtained; P = partial content obtained; N = not obtained. Access route is recorded separately in the protocol.

For G, CUDA = ROCm/HIP; it is a migration analogy and is excluded from CANN/CUDA summaries.

Fig. 9. Access-profile detail placing content-acquisition states beside official-content-detail and source-version-clarity scores. It follows the workflow groups and task order of Figures 6–8 to support interpretation of acquisition outcomes.

Ascend C development page through web_fetch, it obtained only page scaffolding, including navigation, a brief description, and documentation links, without the implementation steps, configuration explanations, or compilation and deployment instructions. web_fetch explicitly reported “No code blocks, no implementation details, no step-by-step commands.” The corresponding CUDA task obtained detailed official content, with M3 = 5. M1 records official-source discovery conditions, M2 records the acquisition outcome, and M3 for the CANN side of D is marked as blocked. This difference highlights the need for official webpages to deliver procedural content in a form that agents can actually read, so that the available technical material can provide a basis for their answers.

Once content is obtained, its task support still differs. Among CANN’s 24 assessable records, ten each have M3 = 4 and M3 = 5, and four have M3 = 2 or 3; all 25 CUDA records have M3 = 4 or 5. In CANN task E, the content the agent obtained through web_fetch from the official EZ9999 explanation listed the cause as “N/A” and suggested “Check whether the CANN package is correct” and “Check whether the environment variable is correct.” It did not

provide specific environment-variable names, checking commands, or criteria for interpreting the results. The content acquired by the agent contained only generic checking suggestions and lacked diagnostic information specific to this error, making it difficult to determine the next action, with M3 = 2. The corresponding CUDA task obtained a checking-tool guide with specific commands, tool options, and report formats, with M3 = 5. This difference concerns whether the acquired material supplies the steps and explanations needed for diagnosis.

Task X, also a debugging task, obtained detailed official guidance in both ecosystems, with M3 = 5 on each side. Comparing E and X locates the issue in specific task content: CANN's memory-checking task already has substantial official guidance, while its error-code diagnosis task lacks information needed to select the next check. The improvement target is the content of error explanations and diagnostic guidance.

Task A shows how useful main-path material and unavailable references can coexist. When the agent read the official conversion quickstart through web_fetch, it obtained an ATC conversion command and parameter explanations, including `--input_shape` for the input shape, `--soc_version` for the target processor, and guidance on querying processor information with `npu-smi info`. This content supplied a basis for the conversion command and its parameter settings. A more extensive ATC/AIPP reference returned navigation and metadata. The worked recording example links the quickstart to supported conversion requirements and the missing reference to unresolved advanced settings. In task Y, accessing chip-migration information through the document center likewise yielded only partial content. These cases motivate recording both the content obtained and its relation to a task requirement, with the access route recorded separately where available. For a developer assessing the conversion guidance, this distinction identifies a supported main procedure and advanced settings that still require reference material. It bounds which parts of a proposed action the acquired sources can substantiate.

The supplement's "Selected web_fetch evidence" provides the URLs, timestamps, requests, and relevant return excerpts for A's quickstart, D's development entry, and E's error entry. These excerpts show the web_fetch content actually received by the agent in each read, providing concrete evidence for the case analysis above.

Acquisition effort can differ even when core content is obtained. Both sides of K obtained official core content, but CANN recorded four fetches and one fetch failure, whereas CUDA recorded two fetches without failure, yielding M8 = 1 and M8 = 4, respectively. These records include additional attempts along resource-entry and acquisition paths in the analysis of knowledge-provision conditions.

Version organization affects a broader range of tasks. Mean M4 is $\bar{x} = 2.88$ for CANN and $\bar{x} = 4.24$ for CUDA. Twelve CANN tasks score 2, spanning conversion, installation, training, operator development, inference, optimization, and debugging. In A, similar quickstarts appear in different version trees, leaving a version choice after content acquisition; its M4 is 2, compared with 4 for the corresponding CUDA task. These differences direct attention to the organization of recommended versions, applicability scopes, and compatibility evidence.

Both sides of I obtained compatibility information, with M4 = 4 and response-version explicitness M9 = 5. The CANN task combined sources about the toolkit, driver, framework, and integration package; the acquired compatibility matrix supplied a basis for connecting these relations. Version assessment can thus identify insufficient selection guidance and record how existing compatibility information supports a task. For ecosystem development, the relevant object is the set of queryable and verifiable relations among resources.

After retrieval is delegated to an agent, the applicability declared by a source still needs comparison with the developer's device and software combination. Source applicability and local environment facts supply different information for this comparison: missing compatibility relations require source investigation, while unspecified installed versions require a local environment check.

6.3 Complementary support and limitations across knowledge sources

Across the 25 task pairs, the mean number of third-party candidate sources per task is $\bar{x} = 3.16$ for CANN and $\bar{x} = 3.44$ for CUDA; five and six tasks, respectively, have only one or two candidates. Complementary support requires task-specific examination. In X, both ecosystems obtained detailed official guidance, with one and three third-party candidates, respectively; I has three and two. Differences in counts do not run in the same direction for every task.

Source records further show that platform distribution and acquisition conditions affect candidate entries. For K, both CANN candidates come from CSDN, while CUDA's four candidates span different platforms. CANN records for N, O, and Q include blocked acquisition of Zhihu content. Platform distribution helps identify source relationships to inspect, while returned content determines whether entries can supply usable explanations. Whether different articles on the same platform are independent still requires content examination.

Source combinations make complementarity between official and community knowledge concrete. CANN task X has few third-party candidates but detailed official guidance; D lacks core official content and also has a low third-party credibility score. E records continued searching for cases after the official error explanation proved insufficient. These cases point, respectively, to experience supplementing official guidance, jointly limited core guidance and alternative support, and a need for diagnostic information from other sources. Ecosystem knowledge development therefore needs to examine which task requirements official resources support and which still depend on discoverable, obtainable practical cases and independent explanations.

Model prior-knowledge estimates provide another support indicator. In the main comparison, mean M7 is $\bar{x} = 2.84$ for CANN and $\bar{x} = 4.56$ for CUDA. CANN tasks D, E, I, M, N, S, and X score 2, while all CUDA tasks score 4 or 5. Reading this estimate alongside source records identifies tasks whose external knowledge support warrants closer inspection. We did not separately test performance without retrieval; the estimate describes the configuration of knowledge support and does not validate independent task completion by the model.

M9 and M10 further record the guidance formed during tasks. For example, X has M10 = 5 in both ecosystems, whereas E has M10 = 3 and M10 = 5, respectively. These response properties and the acquired materials provide joint grounds for inspection, extending analysis from knowledge sources to the version statements and procedural conditions in the resulting guidance.

7 DISCUSSION: DESIGN IMPLICATIONS FOR KNOWLEDGE PROVISION AND AGENT INTERACTION

The comparisons develop an account of ecosystem knowledge support through development scenarios, specific knowledge gaps, and complementary sources. We use these results to discuss development priorities, improvements to official resources and agent feedback, and community knowledge and source inspection. The design illustrations make potential applications concrete; their effects have not been evaluated.

7.1 Setting development priorities through scenarios and knowledge requirements

The scenario comparison identifies limited knowledge support for in-depth development in CANN as an area requiring attention. Examining D, M, and related tasks locates specific needs for implementation guides, procedural detail, and supplementary cases; E also identifies diagnostic knowledge as task content requiring development. This demonstrates the diagnostic value of task-level research: comparing development activities and their knowledge requirements produces specific insights into ecosystem development priorities.

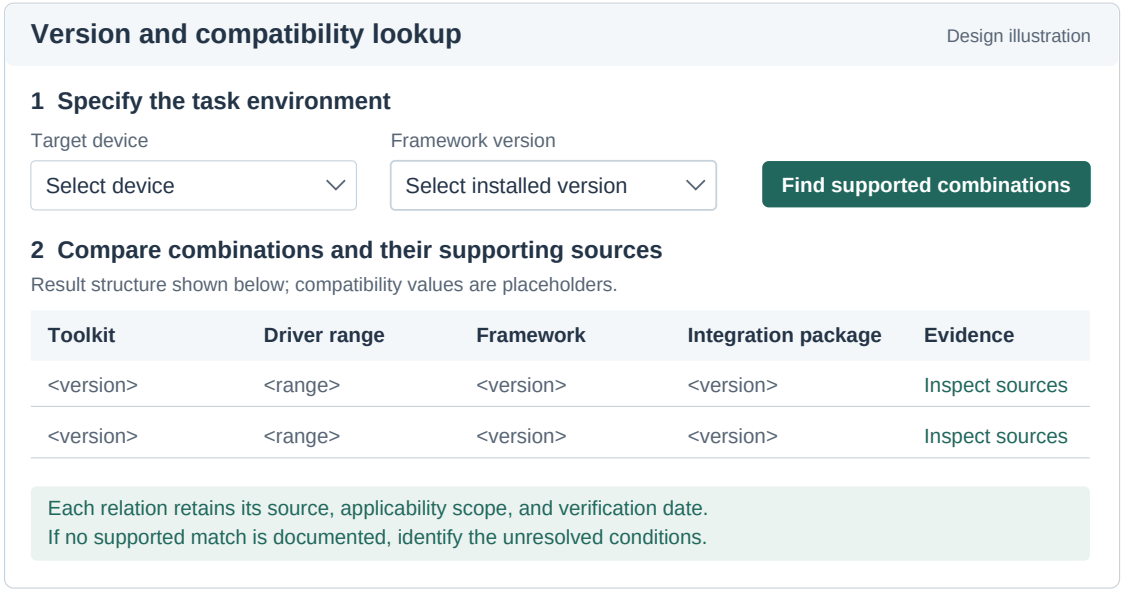


Fig. 10. Proposed version and compatibility lookup, motivated by fragmented version information in A and I. Query constraints, result fields, and source inspection illustrate the proposed information structure; compatibility values are placeholders.

These insights imply different scopes of improvement. Version issues recurring across workflows motivate shared recommended entries and organized compatibility relations; the core content not obtained in D and the limited diagnostic detail in E call for improved content delivery and more complete task guidance, respectively. Tasks with limited third-party support also require examination of practical-case and independent-explanation coverage. Improvements can be checked by returning to the original tasks and examining whether their knowledge requirements receive stronger support.

7.2 Improving official knowledge provision and agent feedback

Recurring version issues suggest treating applicability as a property of a documentation system. A stable recommended-version entry can coexist with clearly indexed documentation for earlier versions, while each page states its applicable versions and verification date. Compatibility relations among drivers, toolkits, frameworks, and integration packages can be published in a queryable form. For a task such as I, this would let a developer or agent request the supported combinations for a stated environment, reducing the need to assemble the relation from several disconnected records.

Figure 10 makes this proposal concrete through a compatibility lookup interface. Device and framework constraints define the query, and each returned combination retains its sources, applicability scope, and verification date. Where no supported match is documented, the interface identifies unresolved conditions. The version problem registered by M4 becomes an inspectable relation, providing a basis for subsequently checking the applicability of a response.

Version-dependent guidance should connect a source's applicability to the available task environment. Before proposing a toolkit/framework combination, an agent can use device and software facts already supplied in the task, requesting only information that remains necessary to establish the match. The response indicators then ask whether the

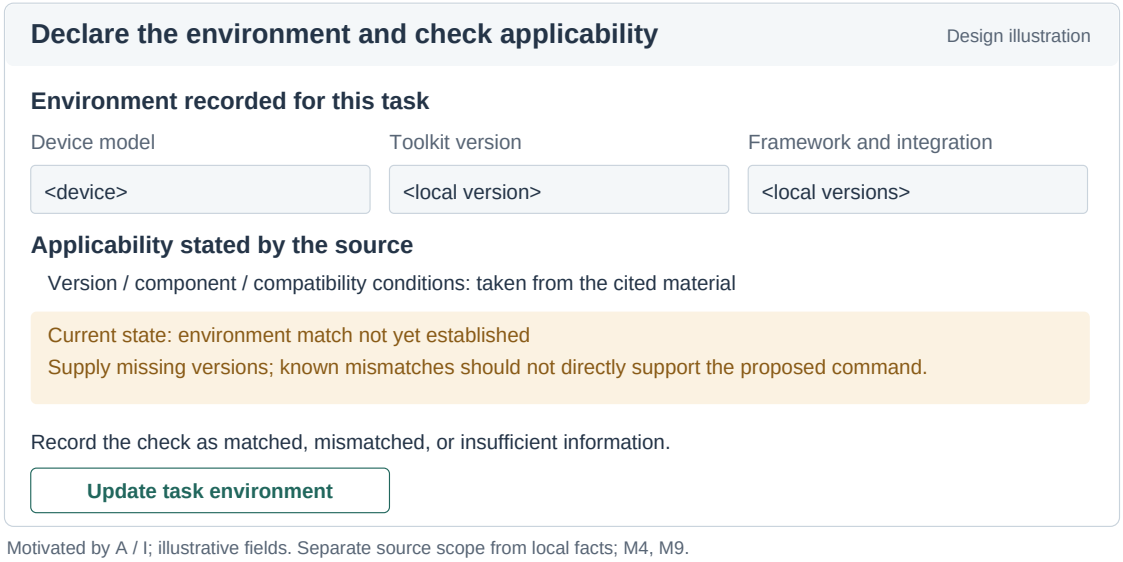


Fig. 11. Proposed environment declaration and version checking, motivated by A and I. Device, toolkit, and framework fields represent developer-supplied local facts, separate from source applicability. The illustrated state has insufficient information to establish a match.

recommendation states its applicable version conditions and provides actionable steps with explicit substitutions or modifications.

Figure 11 illustrates version checking through a separate environment record. It distinguishes developer-supplied local facts from the applicability declared by a source, then records the comparison as matched, mismatched, or insufficient information. The record is reused within the task and updated with additional user input to support applicability checks before specific commands are recommended.

Localized acquisition and content gaps call for task-specific repairs. For the content-acquisition gap illustrated by D, documentation systems should ensure that agents reading official development pages can obtain the complete procedural content. Options include ensuring that HTML pages directly return the full content or providing Markdown or plain-text versions updated alongside the webpages. Whichever approach is used, it should preserve the steps, parameter explanations, and applicable versions needed for the task, with corresponding tasks used to check whether the agent can actually obtain this content. E already has a dedicated error page; its improvement requires more informative content within that entry: error semantics, possible triggers, diagnostic steps, links to issue cases, and applicable versions. For a catch-all error, a guide can state what the code alone leaves unresolved and specify which logs or local observations are needed next. Such a page supports diagnosis by organizing available evidence around the user's problem.

Figure 12 illustrates task-oriented content through E's error page. It starts with what the code can establish, identifies local information to collect, links observable symptoms to checks and attributed cases, and retains a support route for unresolved situations. Domain maintainers would need to establish the actual causes, checks, and cases; the structure helps locate the knowledge that needs to be supplied.

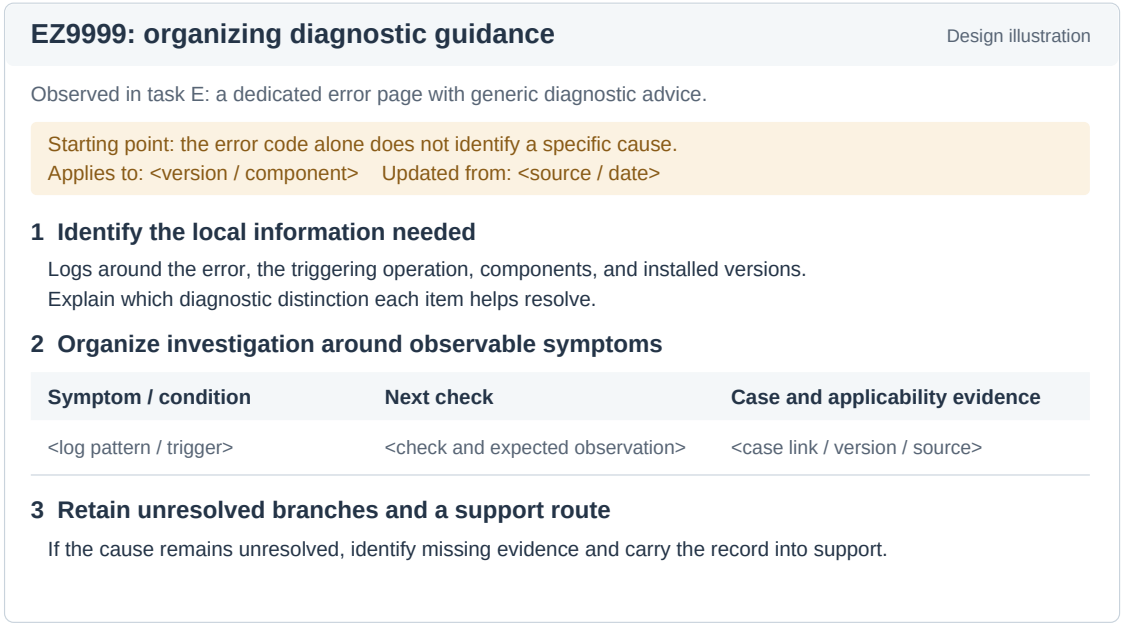


Fig. 12. Proposed task-oriented diagnostic content, motivated by E's dedicated error page with generic guidance. The structure connects a diagnostic starting point, local information, checks, case evidence, and unresolved branches. Specific cases and version scopes remain fields to populate.

The same task-oriented structure can guide other documentation units. An intended outcome, prerequisites, minimal commands or code, parameter explanations, expected output, and recovery information provide concrete material for both human reading and agent retrieval. Machine-readable exports and indexes should preserve those relationships and their version scope. Their success can be checked against the relevant content-acquisition and detail fields for the task that motivated the change.

Figure 13 illustrates delivery of the same task material through a document body and a machine-readable export. Using the command and parameter relations in the conversion quickstart acquired in task A, the export preserves the task, applicable version, source, parameter explanations, and related steps. For a route such as D's, the aim is to deliver the missing core guidance; where a readable body already exists, the aim is to preserve its context during export. Acceptance checks should compare acquired content with task requirements, rather than only checking for a particular file format.

An agent interface can preserve the difference between finding a relevant source, obtaining its body, and supporting a proposed step. For D, an informative status would identify the missing implementation guide. For E, it would indicate that the official explanation is generic and request the logs or trigger context needed for further diagnosis. For A, it could retain the usable conversion path while marking advanced reference settings as unresolved. These statuses connect a retrieval event to its consequence for the developer's current task.

Figure 14 contrasts proposed responses to D and E. For D, the agent first seeks a linked body page or another readable source, while retaining the original entry for optional inspection. For E, it explains how trigger context and relevant

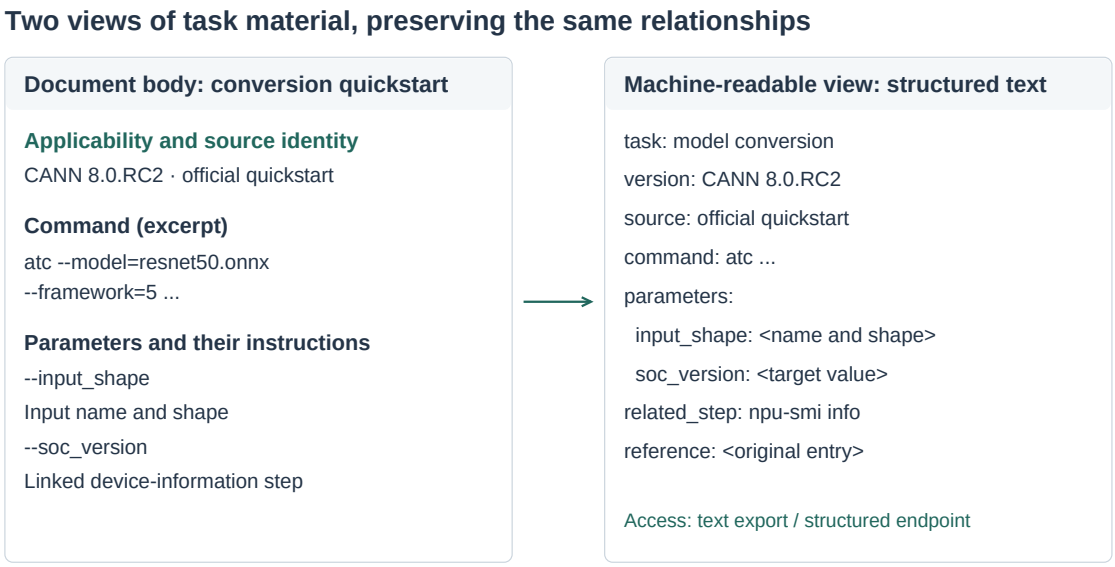


Fig. 13. Proposed document body and machine-readable export. Command, parameter, and device-query relations come from the conversion quickstart acquired in task A; the structured text is a proposed export. Both views preserve the task's version, source, and step relationships.

log excerpts would guide a search for diagnostic cases. Developer-supplied content is a fallback for acquisition failures; accessible core guidance remains a responsibility of the knowledge provider. M1-M3 locate the gap, while M8 records the effort of subsequent attempts.

Knowledge provision also involves entry points and maintenance. Stable, searchable official entries affect whether candidate materials are discovered; explicit ownership, revision dates, and compatibility relations support subsequent assessment. Teams can use representative tasks as recurring inspection units, tracking whether queries reach the intended material, its body is delivered, version relations have changed, and community cases remain applicable. Such maintenance also addresses source governance and third-party knowledge provision beyond the illustrated interfaces.

To address differences in search visibility, documentation teams can inspect search-engine indexing and the alignment of page titles and summaries with development-task terminology, assessing search engine optimization (SEO) improvements through official-result positions and the rounds needed to discover material for task queries.

7.3 Expanding community knowledge and supporting source inspection

Community knowledge development can address task situations that lack adequate support, including specific failures, operating experience under different configurations, and alternative implementations. This direction connects to research on augmenting API documentation with Stack Overflow insights [25]; the present study additionally examines whether agents can discover and acquire such material and whether it supports specific task requirements. Tasks with few candidate sources require checking which supplementary explanations are missing; concentrated or inaccessible

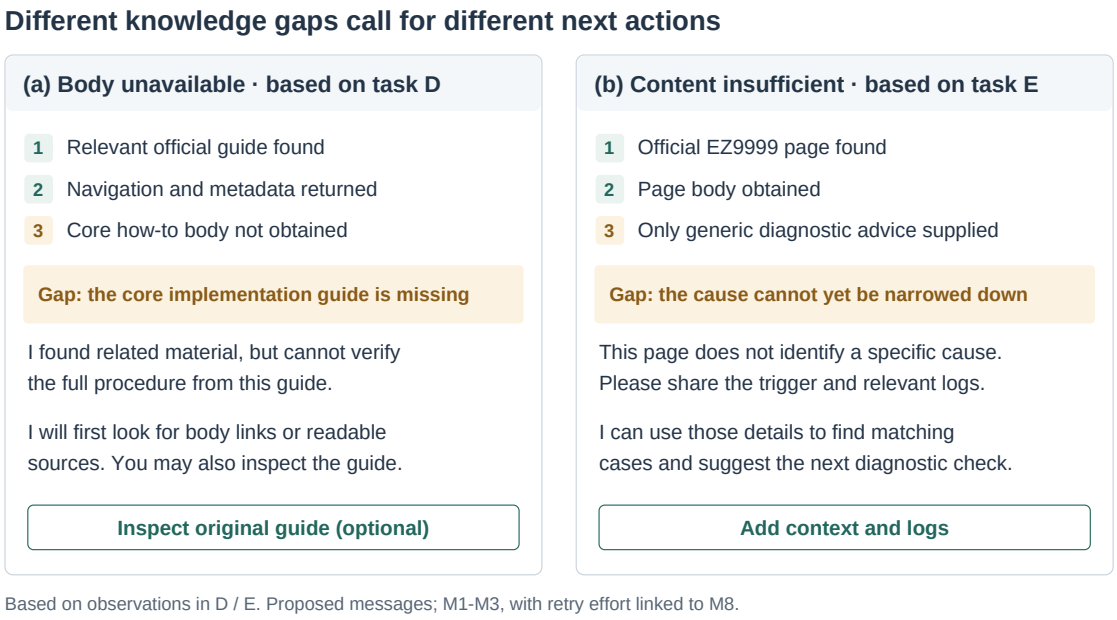


Fig. 14. Proposed agent feedback for two knowledge gaps: (a) a missing core body, based on D; (b) readable but diagnostically insufficient content, based on E. Task observations supply the problem conditions; interface messages and next actions are design proposals.

sources require checking independent practical evidence and content-delivery conditions. The aim is for task-relevant experience to be discoverable, obtainable, and attributable, beyond simply increasing the number of entries.

Consistent with human–AI interaction guidelines on explaining behavior and supporting correction [2], agents can retain source passages, applicability conditions, and their relationships to suggestions to help developers inspect available support. Where content remains missing, they should identify unsupported parts of the task and direct further searches or requests for necessary local information accordingly. The source-inspection and corrective-feedback illustration makes these information relationships concrete.

Because explanations can increase acceptance without improving team performance [3], source presentation should be evaluated by developers' ability to identify unsupported or inapplicable suggestions.

Figure 15 further illustrates source inspection and corrective feedback. A developer can expand the cited passage for a command, inspect its applicable version, and attach an encountered error or correction to the suggestion. The original source, task environment, and new feedback remain connected so the next check can address the specific discrepancy. This provides inspectable support for the response properties described by M9 and M10 and for subsequent revision.

These proposals treat checking suggestions as part of programming work, consistent with the information-seeking and AI-assistance studies discussed earlier [4, 5, 27]. Exposing every retrieval event or source passage could add reading effort without clarifying the next action. An interface could first show the supported step, relevant version scope, and unresolved condition, with source passages available on demand. This prioritizes the information needed for the current judgment while retaining a route to detailed inspection.

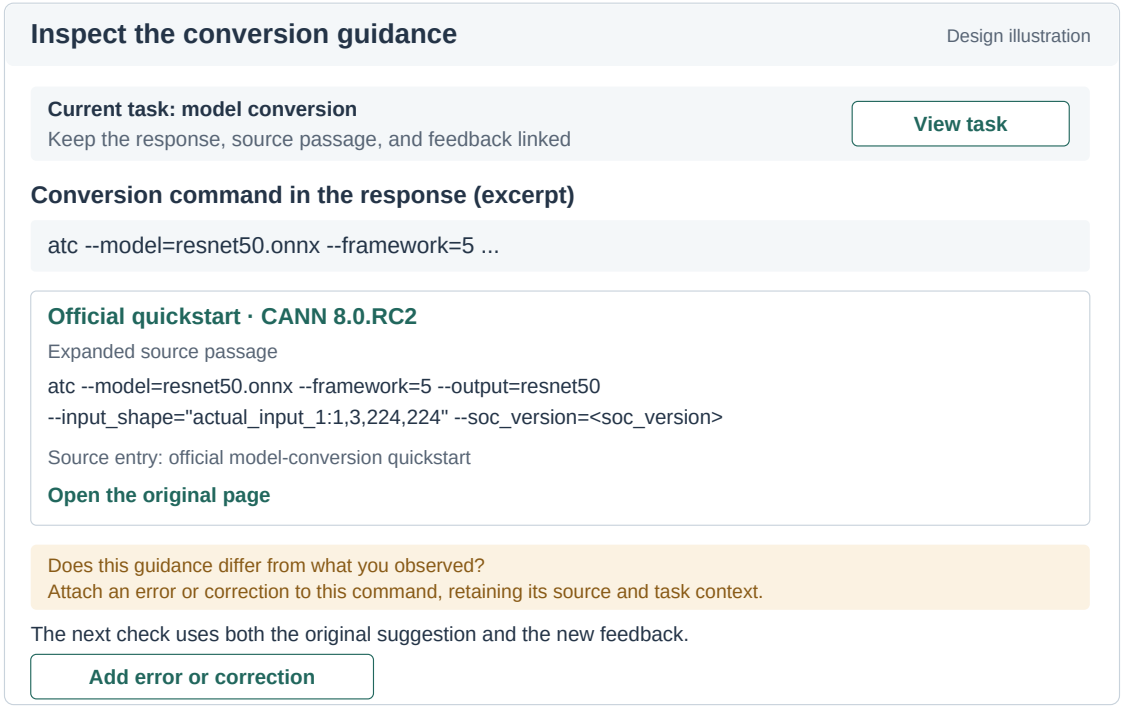


Fig. 15. Proposed source inspection and corrective feedback. The conversion-command excerpt comes from the quickstart acquired in task A; source expansion, an original-page entry, and error feedback linked to the suggestion are proposed interactions.

Requests for environment facts can likewise interrupt a task, especially when they repeat information already provided. A request should explain which recommendation depends on the missing fact and reuse task context where available, with confirmation when it may be stale. For diagnostic logs, it should request relevant excerpts and allow sensitive details to be removed. Missing compatibility relations and generic error explanations require work by documentation and system teams; collecting more local facts cannot supply those missing explanations. The balance between inspection, interruption, and useful guidance remains a question for evaluating these proposals.

7.4 Reusing the diagnostic method

The method connects a task requirement to a source or acquisition event, an indicator code, and a diagnosis of the information needed to assess a proposed action. Professionals such as product managers, designers, technical writers, and software engineers who develop and maintain developer platforms, technical knowledge resources, or AI agents could use the profile to locate missing guidance or applicability relations and distinguish further source retrieval from a request for a local fact. Analysts can reuse the record structure with domain-specific requirements, such as client-server compatibility in a database SDK or deployment prerequisites in a web framework. These are intended applications of the method.

Subsequent evaluation can examine whether independent analysts produce comparable profiles, whether the diagnoses correspond to expert assessments of missing information, and whether documentation teams find them useful for planning repairs. These questions follow directly from the intended use of the method. The present application supplies the task-level distinctions, worked records, and cross-task synthesis on which such evaluation can build.

8 LIMITATIONS AND RESEARCH TRANSPARENCY

The case demonstrates how the method distinguishes breakdowns in discovery, content acquisition, and task adequacy. Task records support inspection of scoring rationales and reproduction of the calculations. The results reflect this task set and its retrieval conditions. Stability across models, tool configurations, and retrieval budgets, as well as independent scoring agreement and applicability across ecosystems, requires further evaluation. The supplement provides observation fields, the scoring implementation, task-to-source mappings, calculation discrepancy notes, and summary scripts to support inspection and subsequent applications.

The study assesses knowledge conditions and recorded guidance. How developers use these distinctions to evaluate evidence, check local applicability, and decide when further information is needed requires observation of their interaction with that guidance. Evaluation should also examine whether the proposed feedback supports those judgments and what reading effort or interruptions it introduces.

9 CONCLUSION

We define knowledge availability for AI through eleven indicators connecting knowledge sources, acquisition processes, and response properties. Task-level comparisons of CANN and CUDA identify differences in knowledge support across development scenarios, including weaker support for in-depth development in CANN, and reveal specific conditions involving content delivery, diagnostic detail, version organization, and supplementary sources. Connecting scores with task requirements and acquired material produces concrete insights into how ecosystems support development activities and which knowledge resources require improvement.

The method makes the basis for AI-mediated technical guidance an object of inspection: which source supports a proposed step, which applicability relation remains unresolved, and what additional information is needed. The framework, protocol, and case analysis provide a way to locate these conditions and relate them to knowledge provision and agent feedback. Documentation repairs, source retrieval, and requests for local facts follow from different diagnoses, offering concrete directions for subsequent evaluation.

APPENDIX A: SCORING CALCULATIONS

Official material (M1–M4). M1 first considers the search process: at least two rounds with query refinement scores 2; at least two rounds without refinement scores 3. For a single round, the first official result at rank 1, 2–6, 7–10, or beyond 10 scores 5, 4, 3, or 2, respectively. Grade 1 describes difficulty finding a relevant official source after repeated searches. Under the study's tool configuration, M2 first considers robots restrictions, then page delivery and observed returns: robots restriction, 1; SPA preventing body retrieval, 2; partial retrieval of task-relevant bodies, 3; retrievable server-rendered body, 4; retrievable static body, 5. Grade 3 means that some targeted body content was obtained while other targeted body content remained technically inaccessible; it does not denote insufficient detail in obtained material or follow automatically from using an alternative entry. M3 is blocked when SPA or robots barriers prevent body acquisition. For acquired content, grade 1 provides no task-specific details, grade 2 only a few task-specific fragments,

and grade 3 an overview or a main path without commands/code. A main path with commands/code or a complete reference without them scores 4; a complete reference with them scores 5. Commands/code denotes operational content in the material. For M4, grade 1 lacks version identifiers; grade 2 lists at least three versions but leaves their compatibility unclear. Where version information is assessable, apply these conditions in order: version-irrelevant task or at most one version, 5; official support matrix, 4; identifiable device–framework pairing, 3; at least three versions with unclear compatibility, 2; remaining cases, 3.

Third-party material (M5–M6). For M5, distinct source counts n of 0, 1–2, 3–4, 5, and at least 6 map to grades 1–5. Official documents, repositories, and forums belong to the official channel. M6 starts with the mean source-type baseline: aggregators or reposts, 2.5; personal technical blogs, 3.0; established Q&A or columns, 3.5; cloud-vendor technical articles or academic papers, 4.0. Three adjustments follow. High, medium, and low content consistency add 1, 0, and −1. Relative to June 2026, median publication ages of at most 36 months, over 36 through 48 months, and over 48 months add 0, −0.25, and −0.5. Platform independence p is distinct platforms divided by source count; $p \geq 0.8$, $0.6 \leq p < 0.8$, and $p < 0.6$ add 0, −0.25, and −0.5. Missing dates are excluded from the median; an entirely missing date or platform field yields zero for that adjustment. Round the result to the nearest integer, resolving half-integer ties to the even integer, then clamp to 1–5.

Model knowledge and acquisition effort (M7–M8). M7 starts from the self-rating anchors in Figure 3. Stable, moderate, and fast technical change add 0, −0.25, and −0.5, respectively; rounding and clamping follow M6. M8 uses weighted effort $c = r + 0.5f + 4e$, where r is search rounds, f fetch calls, and e failed fetch calls. The intervals $c \leq 1.5$, $1.5 < c \leq 2.5$, $2.5 < c \leq 4$, $4 < c \leq 6$, and $c > 6$ map to grades 5, 4, 3, 2, and 1.

Response properties (M9–M10). M9 grade 1 provides no specific version; grade 2 supplies version clues but leaves applicability to the task unresolved; grade 3 specifies a range or some components; grade 4 largely specifies versions; grade 5 gives exact versions. M10 grade 1 offers isolated fragments; grade 2 outlines the main workflow but lacks key implementation details; grade 3 requires minor edits; grade 4 only parameter substitution; grade 5 is directly usable.

Composite confidence (M11). The main-text formula combines official, third-party, and model-knowledge branches, adjusted by M4 and M8. A blocked M3 sets the official branch $O = 0$. M9 and M10 remain separate. The five bands in Figure 5 have lower bounds of 0, 0.24, 0.45, 0.63, and 0.80. Each lower bound is inclusive; each upper bound is exclusive, except that the highest band includes 1.

REFERENCES

[1] Emad Aghajani, Csaba Nagy, Olga Lucero Vega-Marquez, Mario Linares-Vasquez, Laura Moreno, Gabriele Bavota, and Michele Lanza. 2019. Software Documentation Issues Unveiled. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1199–1210. https://doi.org/10.1109/icse.2019.00122

[2] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1–13. https://doi.org/10.1145/3290605.3300233

[3] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1–16. https://doi.org/10.1145/3411764.3445717

[4] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111. https://doi.org/10.1145/3586030

[5] Joel Brandt, Philip J. Guo, Joel Lewenstein, Mira Dontcheva, and Scott R. Klemmer. 2009. Two Studies of Opportunistic Programming: Interleaving Web Foraging, Learning, and Writing Code. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1589–1598. https://doi.org/10.1145/1518701.1518944

[6] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. Association for

Computational Linguistics, 150–158. https://doi.org/10.18653/v1/2024.eacl-demo.16

[7] Claudia Flores-Saviaga, Benjamin V. Hanrahan, Kashif Imteyaz, Steven Clarke, and Saiph Savage. 2025. The Impact of Generative AI Coding Assistants on Developers Who Are Visually Impaired. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1–17. https://doi.org/10.1145/3706598.3714008

[8] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling Large Language Models to Generate Text with Citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398

[9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. 54107–54157. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[10] Diane Kelly. 2009. Methods for Evaluating Interactive Information Retrieval Systems with Users. *Foundations and Trends in Information Retrieval* 3, 1-2 (2009), 1–224. https://doi.org/10.1561/1500000012

[11] Amy J. Ko, Robert DeLine, and Gina Venolia. 2007. Information Needs in Collocated Software Development Teams. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 344–353. https://doi.org/10.1109/ICSE.2007.45

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[13] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Association for Computing Machinery, 1–13. https://doi.org/10.1145/3597503.3608128

[14] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *The Twelfth International Conference on Learning Representations*. https://arxiv.org/abs/2306.03091

[15] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. https://doi.org/10.48550/arXiv.2401.13178 arXiv:2401.13178

[16] Walid Maalej and Martin P. Robillard. 2013. Patterns of Knowledge in API Reference Documentation. *IEEE Transactions on Software Engineering* 39, 9 (2013), 1264–1282. https://doi.org/10.1109/tse.2013.12

[17] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1–16. https://doi.org/10.1145/3613904.3641936

[18] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2785–2799. https://doi.org/10.1145/3576915.3623157

[19] Peter Pirolli and Stuart Card. 1999. Information Foraging. *Psychological Review* 106, 4 (1999), 643–675. https://doi.org/10.1037/0033-295X.106.4.643

[20] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2026. Agentic Much? Adoption of Coding Agents on GitHub. *ACM Transactions on Software Engineering and Methodology* (2026). https://doi.org/10.1145/3822180 Advance online publication.

[21] Martin P. Robillard. 2009. What Makes APIs Hard to Learn? Answers from Developers. *IEEE Software* 26, 6 (2009), 27–34. https://doi.org/10.1109/MS.2009.193

[22] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. Association for Computing Machinery, 491–514. https://doi.org/10.1145/3581641.3584037

[23] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 338–354. https://doi.org/10.18653/v1/2024.naacl-long.20

[24] J. Sillito, G. C. Murphy, and K. De Volder. 2008. Asking and Answering Questions during a Programming Change Task. *IEEE Transactions on Software Engineering* 34, 4 (2008), 434–451. https://doi.org/10.1109/TSE.2008.26

[25] Christoph Treude and Martin P. Robillard. 2016. Augmenting API documentation with insights from stack overflow. In *Proceedings of the 38th International Conference on Software Engineering*. Association for Computing Machinery, 392–403. https://doi.org/10.1145/2884781.2884800

[26] Gias Uddin and Martin P. Robillard. 2015. How API Documentation Fails. *IEEE Software* 32, 4 (2015), 68–75. https://doi.org/10.1109/ms.2014.80

[27] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. Association for Computing Machinery, 1–7. https://doi.org/10.1145/3491101.3519665

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. 50528–50652. https://doi.org/10.52202/079017-1601

[29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629v3

[30] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *The Twelfth International Conference on Learning Representations*. 15585–15606. https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html